AI Code Quality Auto-Evaluation: A Diff-Based Agentic Workflow for SOLID Principle Violation Detection

Jiahe (Jay) Chen · Mentor: Prof. Jiacheng Shen
Duke Kunshan University · Division of Natural and Applied Sciences · Class of 2026

Introduction & Motivation

Modern software increasingly relies on LLMs for code generation. Ensuring adherence to **SOLID design principles** is critical but **manual evaluation does not scale** to production codebases. We automate SOLID violation detection using a diff-based agentic workflow.

SOLID Principles

- **S**ingle Responsibility — one reason to change
- **O**pen / Closed — open for extension, closed for modification
- **L**iskov Substitution — subtypes substitutable for base types
- **I**nterface Segregation — prefer many specific interfaces
- **D**ependency Inversion — depend on abstractions

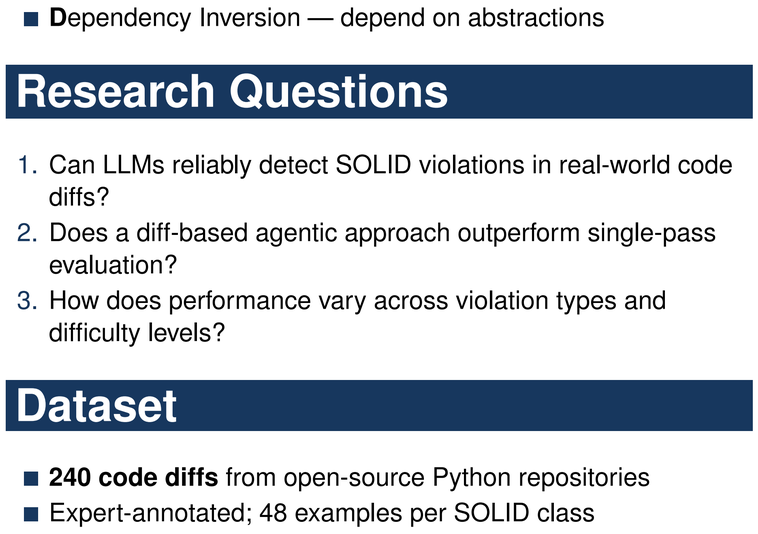
Research Questions

1. Can LLMs reliably detect SOLID violations in real-world code diffs?
2. Does a diff-based agentic approach outperform single-pass evaluation?
3. How does performance vary across violation types and difficulty levels?

Dataset

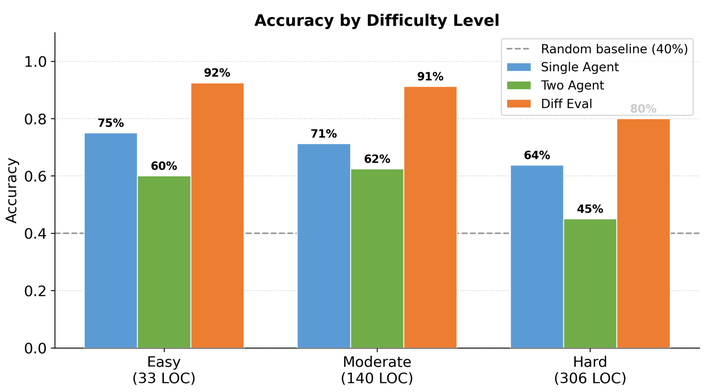
- **240 code diffs** from open-source Python repositories
- Expert-annotated; 48 examples per SOLID class
- Difficulty: Easy (33 LOC), Moderate (140 LOC), Hard (306 LOC)

Results: Overall Accuracy



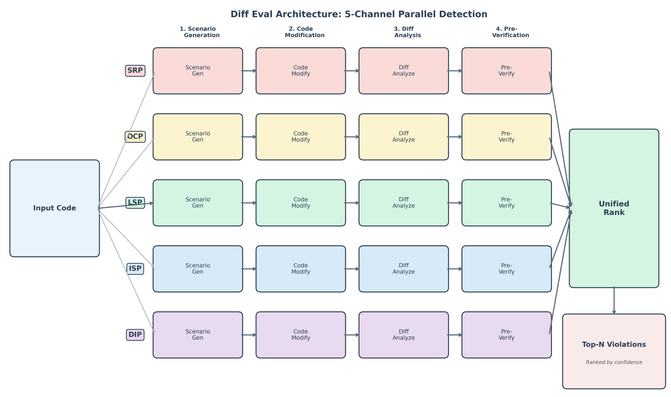
Diff Eval achieves **87.9%**, outperforming Single Agent (70.0%) and Two Agent (55.8%) by **+17.9 pp**.

Results: Accuracy by Difficulty



Diff Eval maintains $\geq$80% even on hard examples (306 LOC).

System Architecture



Five parallel sub-agents each specialise in one SOLID principle. Each channel independently generates scenarios, modifies code, analyzes the resulting diff, and pre-verifies its verdict before a unified ranking resolves conflicts.

Diff-Based Evaluation Pipeline

1. **Scenario Generation** — Enumerate plausible violation scenarios for the target principle.
2. **Code Modification** — Minimally transform code to embody the violation
3. **Diff Analysis** — Confirm structural signals in the resulting diff
4. **Pre-Verification** — Self-consistency check before unified ranking

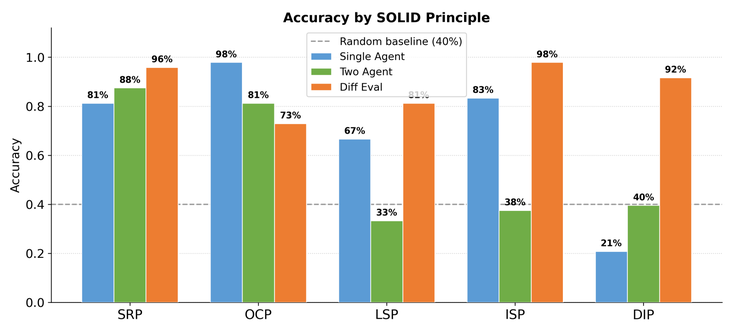
Why Diff-Based?

- Focuses on *changes* introduced by a commit, not pre-existing style
- Reduces false positives; mirrors pull-request code review
- Provides fine-grained, explainable output per principle

Baseline Comparisons

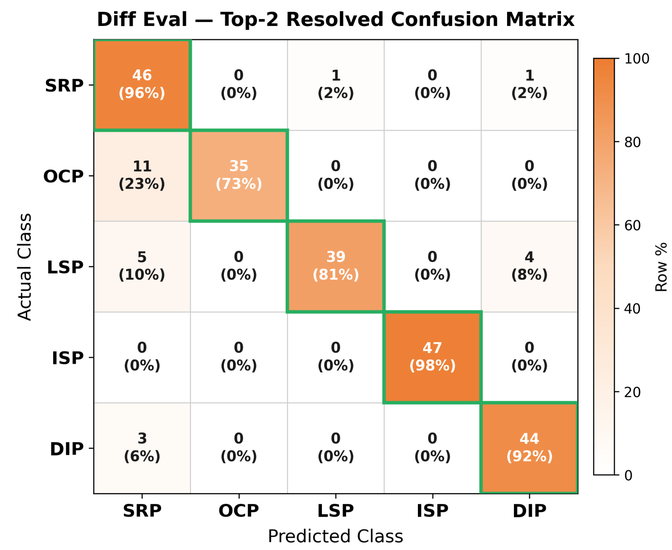
Method	Agents	Diff	Accuracy
Single Agent	1	×	70.0%
Two Agent	2	×	55.8%
Diff Eval	**5**	✓	**87.9%**
Random Baseline	—	—	40.0%

Accuracy by SOLID Principle



ISP (98%) and SRP (96%) reach near-perfect accuracy. LSP (81%) remains the most challenging, reflecting its subtle type-substitution semantics.

Confusion Matrix — Diff Eval



Note: 4 examples with no valid prediction excluded.

Strong diagonal dominance confirms the model reliably distinguishes all five SOLID classes. OCP-to-SRP confusion (23%) is the main error mode, reflecting structural overlap between the two principles.

Key Findings

- Diff-based agentic workflow achieves **87.9% accuracy**, far above the 40% random baseline
- Specialised per-principle agents are critical: the generalist Two Agent baseline drops **14 pp** vs. Single Agent
- Performance is **robust to code length** — hard examples (306 LOC) still reach 80%
- ISP and SRP are easiest; **LSP remains the hardest** class across all methods

Conclusion

We demonstrate that a diff-based multi-agent LLM pipeline can **automate SOLID violation detection at scale**, matching expert-level precision on a diverse benchmark.

The diff-based framing is the decisive design choice: by constraining analysis to code changes, the system focuses on violations *introduced* by a commit, eliminating noise and yielding highly interpretable, actionable reports.

Future Work

- **CI/CD integration** — real-time SOLID checks on pull requests
- Multi-language support beyond Python (Java, TypeScript)
- Extension to broader design pattern detection
- Fine-tuning smaller models on the annotated benchmark

Acknowledgements

This work was conducted under the supervision of Prof. Jiacheng Shen, Division of Natural and Applied Sciences, Duke Kunshan University. The author thanks the DKU Data Science program for computational resources and peer reviewers for feedback.

References

1. Martin, R.C. (2000). *Design Principles and Design Patterns*. Object Mentor.
2. Wei et al. (2022). Chain-of-Thought Prompting. *NeurIPS*.
3. Chen et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.
4. Yao et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*.